

Spec-Driven Development

Making AI Coding Assistants Build What You Actually Want

Dr. Roland Huß

Distinguished Engineer, Red Hat



Dr. Roland Huß

Distinguished Engineer, Red Hat AI
ro14nd.de

Building a platform for running agentic applications
securely on Kubernetes

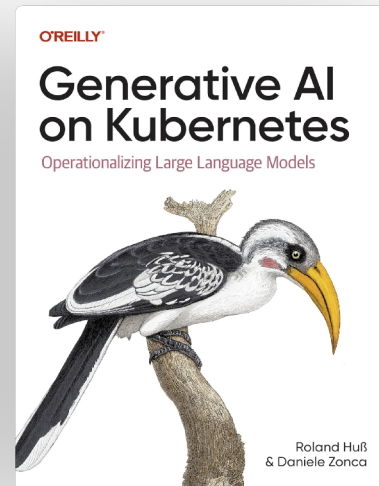
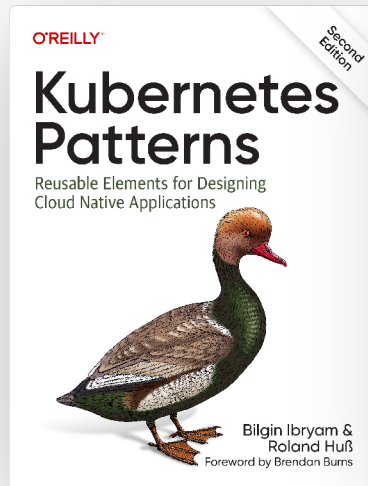
Agentic AI Foundation Ambassador

30+ years of writing code



Generative AI on Kubernetes

Free download from Red Hat
bit.ly/genaik8s-redhat-free



"Add a delete account feature"

What the AI delivers

- Delete button with confirmation
- User record removed from DB
- Compiles, tests pass, PR clean

What it missed

- Subscriptions still billing
- API keys still valid
- Shared resources now ownerless
- OAuth sessions still active

You knew all of this. You just didn't say it.

The bottleneck is no longer writing code. It's articulating your **intent.**

Vibe coding works until it doesn't

What most people do

- Chat with the AI
- Hope it gets it right
- Fix what's wrong
- Repeat

What happens over time

- Context window fills up
- Compaction drops details
- Decisions don't survive
- Memory $\neq$ feature intent

Plan mode?

- Better alignment
- But plan lives in session
- Session ends, plan gone
- Intent stays ephemeral

The problem isn't the AI. The problem is the input.

What if we gave it something durable?

SDD: build the spec first, then let the AI implement

The **specification** is a contract between your intent and the machine's output.

Dialogue, not documentation

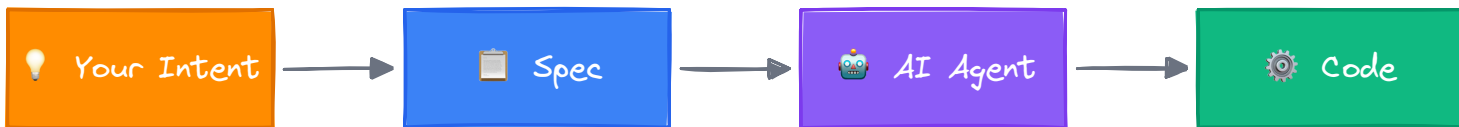
- Co-authored with the AI
- AI asks, you direct
- Spec records decisions

Durable artifacts

- Survives sessions
- Survives context windows
- Shareable across teams

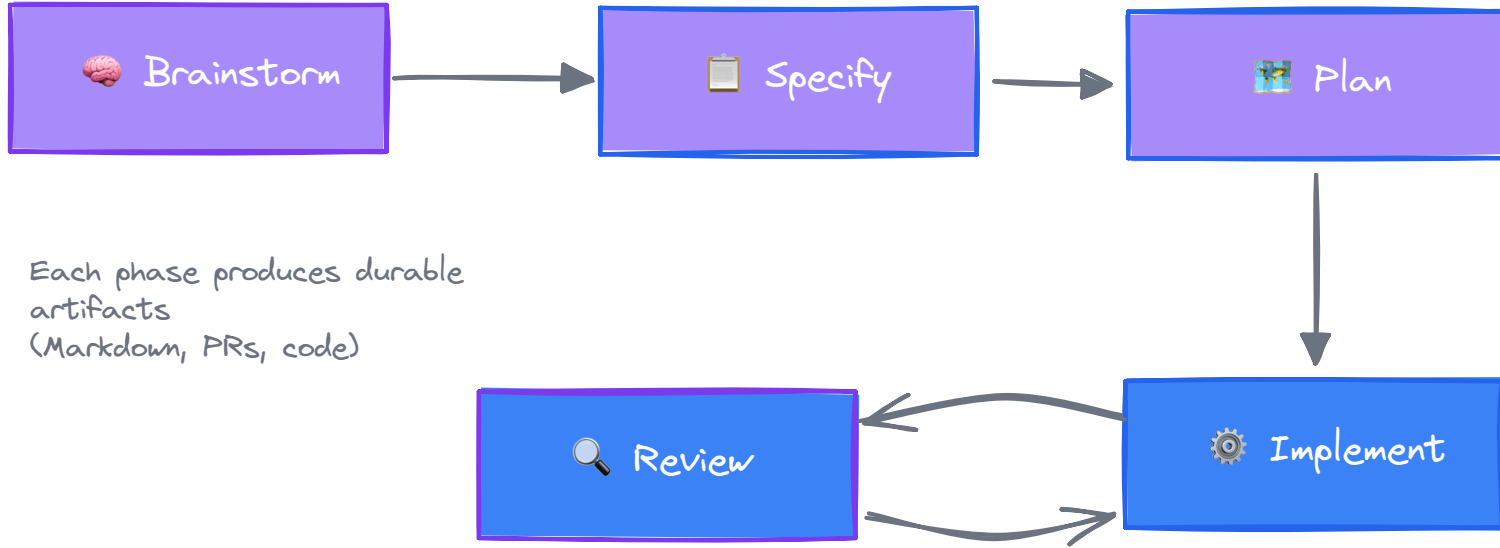
Accurate, not excessive

- Enough for the agent to act
- Not a 50-page requirements doc
- If too long to review, split it



The spec bridges intent and execution

A typical SDD flow



The conversation before coding is where the real work happens.

Multiple tools, same core pattern

Tool	Focus	Approach
Spec Kit	Broad (30+ agents)	Structured workflow, extensible
BMAD	Full lifecycle	Simulated agile team
OpenSpec	Change-focused	Delta specs, anchored
GSD	Solo developers	Minimal overhead
Kiro	AWS ecosystem	IDE-native SDD
Superpowers	Process discipline	Plans + quality gates

Today we'll work with **Spec Kit** + **spex** (composable extensions for collaboration, review, and brownfield support).



The spec replaces your chat message

specs/001-devdash-tui/spec.md

```
1  ### User Story 1 - Survey open work across tracked repositories (Priority: P1)
2
3  A developer opens the dashboard and immediately sees the repositories they track,
4  how many pull requests are open on each, and whether those are passing CI.
5
6  **Why this priority**: This is the entire premise. Without it there is no dashboard.
7
8  **Acceptance Scenarios**:
9  1. **Given** a configured set of tracked repositories and valid credentials,
10     **When** the application launches, **Then** the repository pane lists each
11     tracked repository with its open pull request count and a rolled-up CI indicator.
12
13  **Requirements**:
14  - **FR-001**: The dashboard MUST present two panes simultaneously.
15  - **FR-004**: The CI indicator MUST distinguish four states: passing, failing,
16    pending, and no checks configured.
```

This is what your AI reads instead of "build me a dashboard."

SDD is not waterfall

What people fear

- Spec is frozen upfront
- Can't adapt to changes
- Months between spec and code
- "Big Design Up Front"

How SDD actually works

- Spec evolves with the code
- New requirements? Update and continue
- Minutes between spec and code
- Spec, code, tests stay in sync

The feedback loop is **minutes**, not months.

In a few minutes you'll watch it happen: the plan phase caught a defect in our spec before a line of code existed.

Let's build devdash from scratch

A terminal dashboard: git repos, open PRs, CI status in one view.

The starting prompt

- 1 Build a developer dashboard TUI that shows my git repos,
- 2 open PRs, and CI status in a single terminal view.
- 3 Repos are configurable in a dedicated settings screen
- 4 with drilldown by GitHub organization.
- 5 Use Rust with ratatui.

This is all we start with. The SDD flow does the rest.

The AI helps you think, not just type

 **LIVE DEMO: Brainstorm**

The AI asks clarifying questions. You provide direction. Together you converge.

The contract: everything downstream references this

Branch: 02-after-spec

- **User stories** with P1/P2/P3 priorities
- **Acceptance criteria** in Given/When/Then
- **Functional requirements**, numbered and traceable
- Everything in specs/<feature>/ as Markdown, committed with the code

Every decision traces back to the spec

demo/03-after-plan · demo/04-after-tasks

Plan

- `research.md` — 11 findings, each verified against the live API
- `plan.md` — stack, architecture, phases
- `data-model.md` and 5 contracts
- `quickstart.md`

Tasks

- Ordered and dependency-aware
- Each task references a requirement ID
- Clear start and end conditions

The plan found a bug in the spec. GraphQL caps a page at 100 pull requests, but FR-006 derives repository health from *all* of them. A repo with a failure at position 250 would have shown as passing. Caught before a line of code existed.

The spec guides every line of code

LIVE DEMO: Implementation

The agent picks up tasks from the spec and builds against them.

First: `/clear` to give the agent a fresh context window. The spec and tasks are on disk, not in the conversation.

Review checks the spec, not just the code

Branch: `06-after-review`

- Review output: what passed, what got flagged
- **REVIEWERS.md**: auto-generated review guide
- Questions, not checklists: Why / What / How
- Again: `/clear` first, fresh eyes on the code

Cycle complete: **brainstorm** → **specify** → **plan** → **implement** → **review**

The cycle ran. The tool is still annoying.

Branch: 07-after-evolve

What I said, as a user

- Nothing told me it saved
- No obvious way back to the dashboard
- The repos I picked weren't there

Spec was silent · 2 of 3

No requirement for save feedback. Back-navigation specified one level only.

→ evolve updates the spec

Spec was right · 1 of 3

Its own acceptance test required those repos to appear. The task was checked off half-done.

→ evolve fixes the code

This works for one person. What about teams?

Solo dev is solved

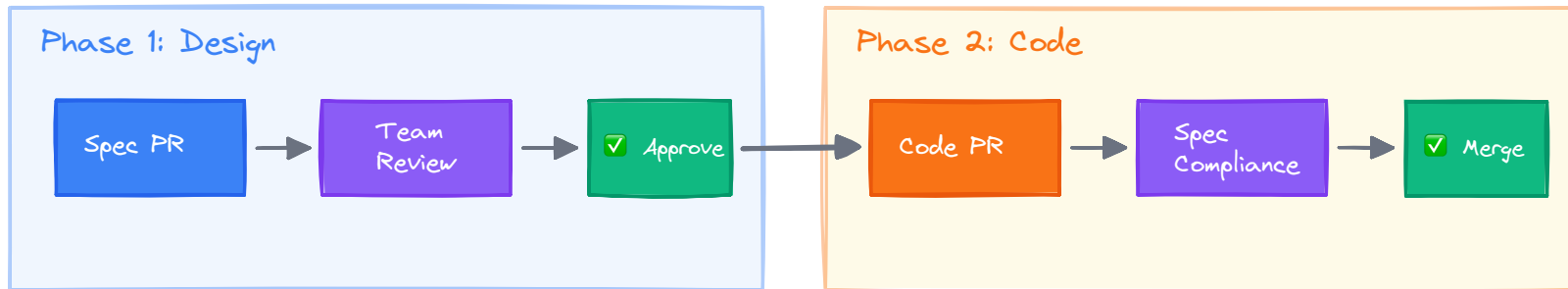
- One spec, one developer
- One feature at a time
- Full cycle works smoothly

Real world is harder

- Multiple people, one codebase
- Who owns the spec?
- How does code review work?
- What about existing repos?

SDD needs to grow beyond solo dev to be useful.

Catch design problems before writing code



- **Phase 1:** Submit the spec for review. Discuss design. No code yet.
- **Phase 2:** Implement against the approved spec.
- Design conversations happen **before** any code exists.

"It works but that's not what I asked for" disappears when intent is documented and reviewed first.

Two reviews, two purposes

Spec Review (human)

- Auto-generated **REVIEWERS.md** as guide
- Is the intent clear? Edge cases covered?
- Design decisions sound?
- Catch problems **before** any code

Code Review (human + agents)

- Humans review architecture, intent alignment
- **Review agents** assist with spec compliance
- Tedious checks increasingly delegated

Humans focus on **design and intent**. Agents handle **compliance and consistency**.

Let agents do the tedious review work

Six-agent deep review

- Correctness
- Architecture
- Security
- Production readiness
- Test quality
- Goal alignment

Autonomous fix loop: agents fix what they find, then re-review.

Integrates with external reviewers

- **CodeRabbit** for automated PR review
- **Codex** for second-opinion analysis
- **Copilot** code review
- Automatic **triage** of bot vs human comments

Also available standalone as `cc-review`.

Use SDD on repos that don't use SDD

The objection

- "My project isn't greenfield"
- "My team doesn't use specs"
- "I can't add spec files to the repo"

Detach mode

- Spec artifacts hidden via `.git/info/exclude`
- Full SDD workflow locally
- Repo only sees your clean code
- When ready, switch to integrated

You get structured development. The repo never knows.

What else is possible

Quality & consistency

- **Quality gates** between phases
- **Spec drift detection** (code vs spec)
- **Constants checking** across artifacts

Scale & automation

- **Team parallelism** via agent teams
- **Worktree isolation** per feature
- **Autonomous pipeline** (brainstorm to merge)

All composable extensions. Start with the core, add what you need.

Three things to remember

1

The spec is the contract

Not documentation. Not planning overhead. It's the input that makes AI useful.

2

Start simple

Brainstorm, specify, implement. Add extensions when you need them.

3

SDD scales

Solo to teams, greenfield to brownfield. The methodology stays the same.

Thank You!

All links & frameworks



bit.ly/sdd-resources

Connect

✉ rhuss@redhat.com

in [linkedin.com/in/ro14nd](https://www.linkedin.com/in/ro14nd)

🌐 ro14nd.de

Questions?